Impresiones por consola de variables/constantes

En el siguiente código se da un ejemplo de impresión de los diferentes tipos de variables

```
.386
.MODEL flat, stdcall
option casemap :none
include \masm32\include\masm32rt.inc
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\masm32.lib

dll_dllcrt0 PROTO C
printf PROTO C : VARARG

.data
__cadenita__ DB "Soy un string", 10, 0
__new_line__ DB 13, 10, 0 ; CRLF

floati DQ 5.33 ; por como funciona printf, hay que si o si declararlas de 64bits
_floati DQ -5.33

inti DW 32767
_inti DW -32768

longi DD 2147483647
_longi DD -2147483648

uinti DW 65535
_uinti DW 0

ulongi DD 4294967295
_ulongi DD 0

shorti DB 127
_shorti DB -128
@varForPrintShort DW ?; por como funciona printf, los short necesitan que se
haga una promoción a int

hexi  DW  0128Fh
octi DW 077o

.code
start:
invoke StdOut, addr __cadenita__

invoke printf, ADDR __cadenita__
INVOKE printf, addr __new_line__

invoke printf, cfm$("%.20Lf\n"), floati
invoke printf, cfm$("%.20Lf\n"), _floati
INVOKE printf, addr __new_line__

invoke printf, cfm$("%hi\n"), inti
invoke printf, cfm$("%hi\n"), _inti
INVOKE printf, addr __new_line__

invoke printf, cfm$("%d\n"), longi
invoke printf, cfm$("%d\n"), _longi
```

```
INVOKE printf, addr __new_line__

invoke printf, cfm$("%u\n"), uinti
invoke printf, cfm$("%u\n"), _uinti
INVOKE printf, addr __new_line__

invoke printf, cfm$("%u\n"), ulongi
invoke printf, cfm$("%u\n"), _ulongi
INVOKE printf, addr __new_line__

MOV AL, shorti
MOVSX AX, AL
MOV @varForPrintShort, AX
invoke printf, cfm$("%hd\n"), @varForPrintShort

MOV AL, _shorti
MOVSX AX, AL
MOV @varForPrintShort, AX
invoke printf, cfm$("%hd\n"), @varForPrintShort

INVOKE printf, addr __new_line__

invoke printf, cfm$("%X\n"), hexi ; X o x, dependiendo si la impresión lleva
mayus o minus
invoke printf, cfm$("%d\n"), hexi ; la hexa interpretada como decimal

INVOKE printf, addr __new_line__

invoke printf, cfm$("%o\n"), octi ;
invoke printf, cfm$("%d\n"), octi ; la octal interpretada como decimal

INVOKE printf, addr __new_line__

end start
```

Notas

- Por como funciona printf, para imprimir los float, es necesario declararlos (o haceres promoción) a 64 bits. Esto no afecta el funcionamiento del resto de las instrucciones o el chequeo de rangos de los float que deban implementar.
- Por como funciona printf, para imprimir shorts, hay que hacerles una promoción a enteros.
- Para imprimir string, pueden usar directamente el printf o el stdout.

Caso especial: Funciones

El printf toca algo de la pila donde está la dirección a donde se debería volver al hacer el RET, es por eso, que al hacer impresiones dentro de una función, van a notar que la ejecución corta. Esto se puede arreglar agregando prefijos y sufijos de código a las funciones que permitan hacer la preservación de la pila y los registros:

Prefijo (se coloca luego del inicio de la función, antes de todas las instrucciones de la misma)
`push ebp ; Save the old base pointer value` `mov ebp, esp ; Set the new base pointer value`

```
sub esp, 4 ; Make room for one 4-byte local variable
push edi ; Save the values of registers that the function
push esi ; will modify. This function uses EDI and ESI
```

Sufijo (se coloca justo antes del RET)

```
pop esi ; Recover register values
pop edi
mov esp, ebp ; Deallocate local variables
pop ebp ; Restore the caller's base pointer value
```

Referencias

- https://cplusplus.com/reference/cstdio/printf/
- https://learn.microsoft.com/en-us/cpp/c-runtime-library/format-specification-syntax-printf-and-wprintf-functions?view=msvc-170
- https://www.cs.virginia.edu/~evans/cs216/guides/x86.html#calling